

ISET Sidi Bouzid

Développement Mobile Android avec Java

Cours

01 – L'ÉCOSYSTÈME ANDROID

Enseignant : Kheireddine Derbali
Année universitaire : 2026/2027
Département : Département Informatique

Année universitaire 2026/2027

1. OBJECTIFS DU COURS

- Comprendre l'architecture globale de la plateforme Android.
- Identifier les composants majeurs d'une application mobile.
- Maîtriser le cycle de vie, la gestion des ressources et le fichier manifeste.
- Comprendre la chaîne de compilation et le format d'exécution final ('.apk').

2. QU'EST-CE QU'ANDROID ?

- **Système d'exploitation** : Développé par Google, basé sur un noyau Linux open-source.
- **Usage** : Smartphones, tablettes, téléviseurs, embarqués.
- **Langages clés** : Java pour le code source de l'application.
- **Environnement** : Android Studio (IDE) + Android SDK (outils de build).

3. ARCHITECTURE SYSTÈME D'ANDROID (EMPILEMENT DES COUCHES)

COUCHE D'ARCHITECTURE	COMPOSANTS & TECHNOLOGIES CLÉS	RÔLE PRINCIPAL
1. Applications Layer (<i>Couche Haute</i>)	• Applications Natives (Navigateur, Contacts) • Applications Tierces (Installées via Store)	Interface utilisateur directe, point d'entrée pour l'utilisateur final.
2. Application Framework (<i>Cadre d'Application</i>)	• Activity Manager (Cycle de vie) • Content Providers (Partage de données) • Resource Manager (Accès ressources) • Notification Manager (Alertes) • View System (Composants UI)	Fournit les API Java réutilisables que les développeurs utilisent pour créer des applications.
3. Platform Libraries & Android Runtime	• Bibliothèques C/C++ : SQLite, OpenGL, WebKit, SSL. • ART (Android Runtime) : Compilation AOT (Ahead-Of-Time). • Remplace Dalvik (JIT).	Offre les fonctionnalités de bas niveau et gère l'exécution optimisée du bytecode (.dex).
4. Noyau Linux (<i>Linux Kernel - Couche Basse</i>)	• Drivers matériels (Caméra, Wi-Fi, Écran) • Gestion mémoire, processus & sécurité (Sandboxing) • HAL (Hardware Abstraction Layer)	Sert de couche d'abstraction entre le matériel physique (Hardware) et la pile logicielle.

DALVIK

DALVIK (ANCIEN) Compilation JIT (Just-InTime) : Traduction en code machine pendant l'exécution.

ART ANDROID RUNTIME

ART (ACTUEL) Compilation AOT (Ahead-Of Time) : Traduction en code machine dès l'installation.

4. LES 4 COMPOSANTS FONDAMENTAUX D'ANDROID

Toute application Android est construite en combinant un ou plusieurs de ces quatre blocs de base. Chaque composant possède son propre rôle et son propre cycle de vie.

COMPOSANT	CARACTÉRISTIQUES & EXEMPLES	RÔLE PRINCIPAL
1. Activity (<i>Activité</i>)	• Écran de l'application avec une interface utilisateur. • Gère les interactions avec l'utilisateur. • Exemple : connexion, accueil, liste de produits.	Gérer un écran et les interactions directes avec l'utilisateur.
2. Service	• Fonctionne en arrière-plan. • Pas d'interface graphique directe. • Exemple : lecture audio, téléchargement.	Exécuter des tâches en arrière-plan.
3. Content Provider (<i>Fournisseur de contenu</i>)	• Gère l'accès aux données. • Permet le partage contrôlé entre applications. • Exemple : contacts.	Partager des données entre applications de manière contrôlée.
4. Broadcast Receiver (<i>Récepteur de diffusion</i>)	• Écoute les événements diffusés. • Réagit aux événements du système. • Exemple : batterie faible, démarrage, SMS reçu.	Réagir automatiquement à certains événements.

5. COMMUNICATION ENTRE COMPOSANTS : LES INTENTS

5.1. INTENT EXPLICITE

Désigne précisément le composant cible à exécuter en fournissant son nom de classe. Utilisé pour la navigation interne.

```

1 Intent intent = new Intent(this,
2   DetailActivity.class);
3 startActivity(intent);

```

5.2. INTENT IMPLICITE

Spécifie une action générale à accomplir. Le système Android cherche l'application adéquate (ex : ouvrir un lien web, passer un appel).

```

1 Intent intent = new
2   Intent(Intent.ACTION_VIEW,
3   Uri.parse("https://google.com"));
4 startActivity(intent);

```

6. LE FICHIER DE CONFIGURATION : ANDROIDMANIFEST.XML

Situé à la racine du projet, ce fichier XML est le carnet d'identité de l'application transmis au système d'exploitation lors de l'installation.

FONCTIONS MAJEURES & STRUCTURE DU MANIFESTE

- **Déclarer les composants** : Toutes les Activités, Services, Broadcast Receivers et Content Providers doivent y être déclarés.
- **Demander les autorisations (Permissions)** : Spécifier les accès matériels/données requis (Internet, GPS, Caméra, Contacts).
- **Configurer l'application** : Nom du package, version, icône, thème graphique, version minimale du SDK Android supportée.

6.1. Code XML

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     package="com.heyzap.ads.example.integration"
4     android:versionCode="1"
5     android:versionName="1.0" >
6     <uses-sdk
7         android:minSdkVersion="8"
8         android:targetSdkVersion="17" />
9     <!-- IMPORTANT: These permissions are required. -->
10    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
11    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
12    <uses-permission android:name="android.permission.INTERNET" />
13
14    <application
15        android:allowBackup="true"
16        android:icon="@drawable/ic_launcher"
17        android:label="@string/app_name"
18        android:theme="@style/AppTheme" android:hardwareAccelerated="true" android:debuggable=
19        "true">
20        <!-- HEYZAP START -->
21        <!-- You need these lines in your Android Manifest. Otherwise, your app will crash. --
22        >
23        <activity android:name="com.heyzap.sdk.ads.HeyzapInterstitialActivity"
24            android:configChanges="keyboardHidden|orientation|screenSize|smallestScreenSize" />
25        <activity android:name="com.heyzap.sdk.ads.HeyzapVideoActivity" android:configChanges=
26            "keyboardHidden|orientation|screenSize|smallestScreenSize" />
27        <receiver android:name="com.heyzap.sdk.ads.PackageAddedReceiver">
28            <intent-filter>
29                <data android:scheme="package"/>
30                <action android:name="android.intent.action.PACKAGE_ADDED"/>
31            </intent-filter>
32        </receiver>
33        <!-- HEYZAP END -->
34
35        <activity
36            android:name="com.heyzap.ads.example.integration.HeyzapActivity"
37            android:configChanges="orientation|keyboardHidden|screenSize"
38            android:label="@string/app_name"
39            android:theme="@style/AppBaseTheme" >
40            <intent-filter>
41                <action android:name="android.intent.action.MAIN" />
42
43                <category android:name="android.intent.category.LAUNCHER" />
44            </intent-filter>
45        </activity>
46    </application>
47</manifest>

```

7. GESTION DES RESSOURCES & LA CLASSE AUTO-GÉNÉRÉE R.JAVA

Les ressources non-code sont stockées dans le dossier `res/` :

- `res/layout/` : Fichiers XML définissant l'interface utilisateur.
- `res/values/` : Fichiers XML de constantes (`strings.xml`, `colors.xml`).
- `res/drawable/` : Images (PNG, JPEG) et graphiques vectoriels XML.
- `res/mipmap/` : Icônes d'application selon la densité d'écran.

8. CHAÎNE DE COMPILATION ET BUILD ANDROID

Le processus complet qui transforme le code source écrit par le développeur en un fichier d'installation final exécutable par l'appareil mobile.

ÉTAPE DU PROCESSUS	DESCRIPTION DÉTAILLÉE DU TRAITEMENT
1. Compilation Java	Le compilateur <code>javac</code> transforme les fichiers de code source Java (<code>.java</code>) en bytecode Java standard (<code>.class</code>).
2. Conversion DEX	L'outil <code>d8</code> transforme les fichiers <code>.class</code> en fichiers Dalvik Executable (<code>.dex</code>), optimisés pour l'exécution sur Android.
3. Empaquetage (APK)	L'outil <code>AAPT2</code> compile et traite les ressources de l'application. Avec les fichiers <code>.dex</code>, le fichier <code>AndroidManifest.xml</code> et les ressources, ces éléments participent à la génération du fichier final <code>.apk</code> (Android Package).

9. POINTS CLÉS À RETENIR

Noyau Android

Basé sur le noyau Linux (gestion mémoire, drivers, isolation de sécurité).

Moteur d'exécution

ART utilise la compilation AOT (Ahead-Of-Time), plus performante que Dalvik (JIT).

4 Composants

Activity (UI), Service (Background), Content Provider (Partage données), BroadcastReceiver (Événements)

AndroidManifest.xml

Obligatoire. Déclare les 4 composants et les autorisations ..